FORZY

Documentação final da solução - Sprint 04 Generative AI & Advanced Nets

Assistente conversacional para análise de motores, com classificação de falhas, detecção de anomalias e recuperação de documentos.

Versão revisada para entrega - 4 de novembro de 2026.

Os dados são sintéticos. Os resultados apresentados foram verificados em ambiente local e não representam validação industrial.

Equipe

Integrante	RM
Lucca Phelipe Masini	564121
Luiz Henrique Poss	562177
Igor Paixão Sarak	563726
Bernardo Braga Perobeli	562468
Felipe Stefani Honorato	563380

1. Visão geral da solução

A solução reúne informações que normalmente precisam ser consultadas separadamente: leituras de sensores, resultados dos modelos e documentação. O objetivo é ajudar equipes de operação e manutenção a investigar comportamentos fora do padrão e entender os limites de cada conclusão.

O usuário pode consultar um motor em um período, verificar anomalias, analisar uma nova leitura e pedir explicações apoiadas nos documentos disponíveis. Reduzir tempo de investigação e priorizar inspeções são benefícios esperados; esta entrega não mede redução de custos nem prevenção de falhas reais.

Componente	Situação na entrega
Classificação de falhas	Modelo e código reaproveitados da Sprint 02.
Detecção de anomalias	Baseline por motor e Isolation Forest reaproveitados da Sprint 03.
Assistente e RAG	Interface integrada e busca documental implementadas na Sprint 04.
Manutenção realizada	Não há registros de serviços ou trocas de peças na base.
Manuais dos fabricantes	Não foram fornecidos. O guia de interpretação é material da equipe.

O enunciado da Sprint 04 pede que o estado real do projeto seja documentado. Por isso, esta entrega separa funcionalidades implementadas, resultados verificados e melhorias futuras. Não se presume que a ausência de RAG nas sprints anteriores seja erro da equipe ou da avaliação: os enunciados anteriores não foram analisados.

2. Dados utilizados e tratamentos

A fonte operacional é motor.db, um banco SQLite com 20 motores e 30.000 leituras, sendo 1.500 por motor. O período é de 01/01/2026 às 00h00 até 02/01/2026 às 00h59. As leituras têm intervalo de um minuto. Os horários não têm fuso informado e são utilizados na interface como aparecem na base.

Campo	Descrição e unidade
motor_id	Identificador do equipamento; seleciona o cadastro e a referência normal.
timestamp	Data e hora da leitura; define ordem e recorte temporal.
rotacao_rpm	Rotação do eixo, em RPM.
vibracao_mm_s	Vibração, em milímetros por segundo.
temperatura_c	Temperatura, em graus Celsius.
corrente_a	Corrente elétrica, em ampères.
falha	Rótulo histórico: 0 normal; 1 desbalanceamento; 2 superaquecimento; 3 falha mecânica.

A tabela motores registra fabricante, modelo, potência e ano de instalação. A tabela tipos_falha descreve as quatro classes. Existem 26.291 leituras normais, 1.205 de desbalanceamento, 1.718 de superaquecimento e 786 de falha mecânica.

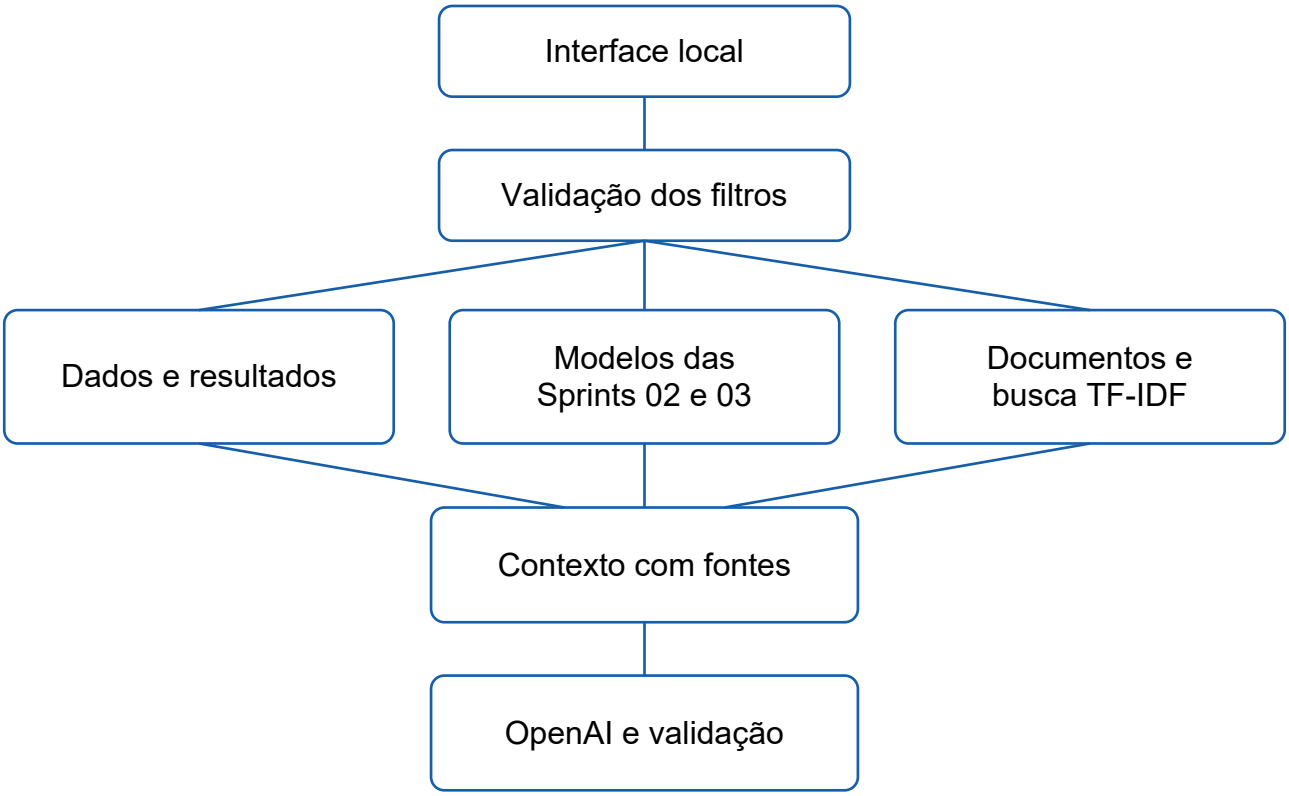
Qualidade e preparação

A conferência encontrou 0 valores ausentes nos quatro sensores e 0 pares de motor e instante duplicados. Não houve valores negativos de rotação, vibração ou corrente. Os 29.980 intervalos entre leituras consecutivas medem 60 segundos.

Os registros são ordenados por motor e instante. O classificador padroniza as quatro variáveis antes do treinamento. O detector aprende a referência normal apenas com leituras normais da partição de treino. Não foi necessário preencher dados ausentes ou remover duplicatas na base entregue. Uma nova leitura com valor não finito é rejeitada.

A simulação inclui ruído e rampas de deterioração. O rótulo de falha aparece após 40% da intensidade da rampa; por isso, um desvio pode surgir antes do rótulo. Não existem ordens de serviço, técnicos ou histórico de troca de peças. Os documentos do RAG descrevem a base e os modelos, e não comprovam procedimentos de um fabricante.

3. Arquitetura da solução



A interface envia a pergunta, o motor e o período ao servidor local. operacao.py valida os filtros, consulta o banco em modo de leitura e reúne os resultados. Quando necessário, também executa o classificador e o detector para uma leitura.

recuperacao.py busca trechos nos documentos. assistente.py junta esses trechos aos resultados operacionais e pede uma resposta à OpenAI. Antes da apresentação, os identificadores das fontes são conferidos. A resposta retorna à interface com as evidências consultadas.

O projeto usa Python, SQLite, pandas, NumPy, scikit-learn, joblib, pypdf e o SDK da OpenAI. A interface utiliza HTML, CSS e JavaScript, sem compilação. A chave fica no servidor; não é enviada ao navegador. Nenhuma resposta aciona fisicamente os equipamentos.

A validação repetiu os três treinamentos e reproduziu as matrizes de confusão originais. As métricas completas acompanham o código. No processamento posterior, janelas de cinco leituras resumem a persistência dos scores; elas não são entradas dos detectores.

4. Modelos de inteligência artificial

Classificação de falhas - Sprint 02

O classificador padroniza os quatro sensores com StandardScaler e usa ExtraTrees: 500 árvores, mínimo de três amostras por folha e semente 43. Os pesos balanceados compensam a menor presença das classes de falha.

A divisão cronológica por motor reserva 21.000 leituras para treino (70%) e 9.000 para teste (30%). Foi mantida a versão anterior de maior recall das falhas. O teste não representa validação externa independente dessa seleção.

Classe	Precisão	Recall	F1
Normal	0,9852	0,9772	0,9812
Desbalanceamento	0,8828	0,8662	0,8744
Superaquecimento	0,7589	0,8629	0,8076
Falha mecânica	0,8680	0,9007	0,8840

A acurácia foi 0,9638 e o F1 macro foi 0,8868. Precisão indica a proporção de acertos entre os casos classificados em uma classe; recall indica quanto dos casos reais dessa classe foi encontrado. F1 equilibra as duas medidas.

Referência normal e anomalias - Sprint 03

A divisão cronológica por motor usa 16.800 registros no treino (56%), 4.200 na validação (14%) e 9.000 no teste (30%). O ajuste usa somente exemplos normais. Os rótulos selecionam esse treino e servem à calibração e à avaliação; não são atributos de entrada.

O baseline resume os desvios dos quatro sensores em relação à média e ao desvio-padrão normal do motor. O Isolation Forest usa 400 árvores, semente 42 e oito entradas normalizadas: quatro desvios assinados e seus valores absolutos.

Método	Precisão	Recall	F1
Baseline por motor	0,9159	0,9553	0,9352
Isolation Forest	0,8182	0,9751	0,8898

A validação escolheu os limiares de maior F1: 2,4666 no baseline e 0,0854 no Isolation Forest. O baseline é a saída principal por precisão e F1; o Isolation Forest tem maior recall e mais falsos positivos. A PR-AUC foi 0,9853 e 0,9370, respectivamente.

5. Recuperação de documentos - RAG

RAG é a geração apoiada em informações recuperadas. A busca seleciona trechos, e a OpenAI redige a resposta usando esses trechos e os resultados calculados localmente.

Há 5 documentos e 17 fragmentos: o PDF do banco, BANCO.md, DADOS_FORZY.md, COMPARACAO_SPRINTS.md e o guia da equipe. Todas as origens são identificadas; o guia não é um manual de fabricante.

Etapa	Como funciona
Preparação	PDFs têm o texto extraído com a página de origem. Textos e Markdown são lidos em UTF-8.
Fragmentação	Trechos de até 180 palavras, com 30 palavras de sobreposição para preservar contexto.
Indexação	TF-IDF atribui pesos a palavras e pares de palavras. O índice é local e fica em memória.
Busca	Similaridade de cosseno seleciona até cinco trechos com pontuação mínima de 0,035.
Metadados	Arquivo, título, página, identificador e hash ligam o trecho à versão da fonte.
Geração	gpt-4.1-mini é o padrão configurável. A resposta usa o contexto fornecido e uma estrutura JSON validada.

O filtro operacional aplica motor e período às leituras. O manifesto documental pode restringir uma fonte a motores e datas de vigência. Documentos sem restrições são gerais. Essa vigência é diferente da data da leitura. A filtragem documental foi testada com arquivos de teste; o corpus inicial é geral.

As instruções de geração exigem português-BR, uso das evidências, distinção entre previsão e diagnóstico e reconhecimento de informação ausente. Também proíbem inventar manuais, serviços ou limites técnicos. A instrução completa está em assistente.py.

O1 identifica a fonte operacional, C1 identifica uma leitura fornecida pelo usuário e F1 em diante identifica os documentos daquela resposta. A interface permite abrir cada fonte. Identificadores inexistentes são rejeitados, inclusive em citações agrupadas como [O1, F2]. Essa verificação não garante, sozinha, que toda conclusão seja correta.

A opção por busca local reduz dependências e facilita conferir as fontes deste corpus pequeno. Não há embeddings remotos nem banco vetorial externo. A busca lexical pode falhar em paráfrases, e sua pontuação não é uma probabilidade de acerto. PDFs sem camada de texto exigem OCR antes da inclusão.

6. Uso do assistente

1. Selecione um motor ou a opção de todos os motores. 2. Informe início e fim nos filtros. 3. Escreva a pergunta ou escolha um exemplo. 4. Leia a resposta e abra as fontes em Evidências. 5. Se desejar, exporte a conversa em JSON.

Para uma nova leitura, abra Nova leitura e informe rotação, vibração, temperatura e corrente. A aplicação executa os dois tipos de modelo e apresenta os resultados junto do contexto. Nova conversa apaga o histórico mantido pelo navegador.

Os filtros definem o período consultado. Expressões como ontem não são convertidas automaticamente em datas. Se a pergunta mencionar um motor diferente do filtro, a aplicação pede correção. Se não houver dados no recorte, ela não substitui o período por outro silenciosamente.

Exemplos atendidos: investigar anomalias do motor 3; comparar seus sensores com a referência normal; explicar os sinais descritos na documentação; e consultar um serviço de manutenção. Neste último caso, a resposta informa que o histórico não existe na base, em vez de inventar uma intervenção.

A aplicação mantém até seis mensagens como contexto da conversa. A severidade mostrada é uma regra do projeto baseada no score. Ela não equivale a uma classificação validada de risco industrial. Procedimentos físicos dependem de documentação adequada e avaliação de um profissional responsável.

7. Testes e resultados

A revisão cobre testes locais, reprodução dos modelos, conferência dos dados, execução dos três cenários exigidos e inspeção da interface. As respostas abaixo foram efetivamente obtidas da OpenAI e estão preservadas em evidencias/cenarios.json. Novas execuções podem produzir redações diferentes.

Cenário 1: falha ou anomalia

Entrada: O motor 3 apresentou anomalias no período selecionado? Informe a quantidade de leituras, o total de anomalias do baseline, o maior score e diferencie isso da classe prevista na última leitura.

Recorte: motor 3, de 01/01/2026 às 17h30 até 02/01/2026 às 00h59.

Resultado esperado: Citar os totais corretos do recorte e separar rótulo, previsão e score.

Resultado obtido

Sim, o motor 3 apresentou anomalias no período de 2026-01-01 17:30:00 a 2026-01-02 00:59:00, com um total de 450 leituras. Foram detectadas 2 anomalias pelo método baseline e 4 anomalias pelo método Isolation Forest, sendo 4 anomalias únicas no total. O maior score de anomalia registrado foi 6,1552, que indica um desvio relevante em pelo menos uma leitura. Entretanto, a última leitura do período não foi classificada como anômala: a classe prevista foi "Normal" com alta probabilidade (0,9876), score de anomalia baixo (0,8838) e severidade considerada baixa. Isso mostra que, apesar das anomalias detectadas ao longo do período, a condição mais recente do motor está dentro do padrão esperado.

Fonte	Origem consultada
O1	motor.db / anomalias_leituras.csv
F4	interpretacao_modelos.md

Análise

Os números correspondem ao recorte: 450 leituras, 2 alertas do baseline, 4 do Isolation Forest e 4 leituras anômalas únicas. O maior score foi aproximadamente 6,1552. A última leitura, isoladamente, foi normal segundo o classificador e ficou abaixo do limiar do detector. Portanto, a condição final não apaga os alertas anteriores.

Registro de execução: 04/11/2026 às 19:12:06 (horário de Brasília).
Modelo retornado: gpt-4.1-mini-2025-04-14. Duração: 3,161 segundos.

Cenário 2: dados e documentação

Entrada: Relacione os dados operacionais do motor 3 com a documentação sobre superaquecimento e explique como investigar os desvios. Use fontes dos dados e dos documentos, sem inventar um manual de fabricante.

Recorte: motor 3, de 01/01/2026 às 17h30 até 02/01/2026 às 00h59.

Resultado esperado: Combinar evidência operacional e documento recuperado, com fontes dos dois tipos e limites da análise.

Resultado obtido

O motor 3, fabricado pela Toshiba, modelo EQP-Global, com potência nominal de 11 kW, apresenta dados operacionais no período de 2026-01-01 17:30:00 a 2026-01-02 00:59:00, com 450 leituras. As leituras de sensores indicam temperatura média de 62,15 °C e máxima de 69,24 °C, e a última leitura apresenta temperatura de 65,59 °C com classificação prevista como "Normal" e baixa severidade de anomalia (score 0,88 vs limiar 2,47) [O1]. Sobre superaquecimento, o classificador ExtraTrees considera temperatura, rotação (RPM), vibração e corrente, atribuindo probabilidades a classes como superaquecimento e desbalanceamento, mas a probabilidade de superaquecimento na última leitura é baixa (0,0046) e o score de anomalia não ultrapassa o limiar [F1]. Para investigar desvios que possam indicar superaquecimento, recomenda-se analisar o motor, período e unidades, verificando a qualidade dos registros; comparar sensores com a referência normal do equipamento; avaliar se o desvio persiste em várias leituras; e confrontar as hipóteses do classificador com a documentação disponível, seguindo o roteiro analítico acadêmico descrito, sem executar intervenções físicas sem profissional [F2]. Assim, não há indicativo atual de superaquecimento, e a investigação deve focar no monitoramento contínuo dos desvios e comparação com a referência normal desse motor.

Fonte	Origem consultada
O1	motor.db / anomalias_leituras.csv
F1	interpretacao_modelos.md
F2	interpretacao_modelos.md

Análise

A resposta reúne a fonte operacional e documentos recuperados. A explicação é limitada à base sintética e ao guia da equipe, sem alegar que houve consulta a manual de fabricante. O roteiro orienta a investigação, sem prescrever intervenção física.

Registro de execução: 04/11/2026 às 19:12:09 (horário de Brasília).
Modelo retornado: gpt-4.1-mini-2025-04-14. Duração: 2,804 segundos.

Cenário 3: informação insuficiente

Entrada: Qual foi a última troca de rolamento do motor 3, em que data e qual técnico realizou o serviço?

Recorte: motor 3, de 01/01/2026 às 17h30 até 02/01/2026 às 00h59.

Resultado esperado: Declarar que não há histórico de manutenção e não inventar data ou técnico.

Resultado obtido

Não há informação disponível sobre a última troca de rolamento do motor 3, nem a data ou o técnico responsável pelo serviço, pois o banco não contém dados de históricos de manutenção, ordens de serviço ou nomes de técnicos.

Fonte	Origem consultada
O1	motor.db / anomalias_leituras.csv
F1	interpretacao_modelos.md

Análise

A resposta reconhece a ausência do histórico e não inventa data, técnico ou troca de peça. O teste demonstra esse comportamento no exemplo executado, sem garantir ausência de alucinações em qualquer pergunta.

Registro de execução: 04/11/2026 às 19:12:10 (horário de Brasília).
Modelo retornado: gpt-4.1-mini-2025-04-14. Duração: 1,17 segundos.

Outras verificações

A revisão atual ampliou a suíte para 12 testes locais. Eles cobrem períodos inválidos, motor inexistente, falta de leituras, valores não finitos, divergência de motor, restrições documentais, busca sem correspondência, citações inexistentes e contagem sem duplicidade entre detectores.

A interface foi verificada em Chromium nas larguras de 320, 768, 1.024 e 1.440 pixels, sem rolagem horizontal. A consulta recebeu HTTP 200 e exibiu as fontes, sem erros de console. As capturas estão em evidencias/. Foram mantidas fora do corpo deste PDF para preservar a padronização tipográfica e de cores solicitada.

8. Instalação, preparação e execução

Requisitos: Python 3.13, acesso à internet para instalar dependências e consultar a OpenAI, e uma chave válida para o projeto escolhido. O uso da API depende do acesso ao modelo e da disponibilidade de saldo ou limite. Os testes locais e a preparação dos modelos não exigem chamada à API.

Obter o projeto

Clone o repositório público indicado na seção 10. Ele contém o código e a base necessários para preparar os modelos no computador de quem executará o projeto, sem depender de arquivos existentes no computador da equipe.

Instalar as dependências

python -m venv .venv

.\.venv\Scripts\python.exe -m pip install -r requirements-lock.txt

Configurar a chave

Copie .env.example para .env.local e preencha OPENAI_API_KEY de forma privada. OPENAI_MODEL é opcional e tem gpt-4.1-mini como padrão. A chave não deve entrar no GitHub, em capturas ou em registros de execução.

Preparar e iniciar

.\.venv\Scripts\python.exe preparar_projeto.py

.\.venv\Scripts\python.exe indexar_documentos.py

.\.venv\Scripts\python.exe app.py

Após iniciar o servidor, abra http://localhost:8501 no navegador.

Os comandos acima preparam os modelos ausentes, indexam os documentos e iniciam o servidor local, nessa ordem. Para encerrar, pressione Ctrl+C no terminal.

Para testar, execute python -m unittest discover -s testes -v pelo Python do ambiente virtual. avaliar_cenarios.py faz três chamadas reais à API, pode ter custo e atualiza suas evidências. Não é necessário executá-lo para simplesmente abrir o projeto.

Se a chave não for aceita, confira a configuração privada. Se houver erro de saldo ou limite, confira o projeto OpenAI. Se faltarem os modelos ou o CSV de resultados, execute preparar_projeto.py. Para mudar a porta, use app.py --porta 8502. As versões completas do ambiente testado estão em requirements-lock.txt.

9. Limitações e continuidade

A solução é acadêmica e usa dados simulados de um período curto. Ainda não há validação em equipamentos reais, ingestão contínua, ordens de serviço, autenticação de vários usuários ou integração para executar ações físicas. Não se deve tratar uma previsão como diagnóstico confirmado.

Os documentos disponíveis explicam os dados e os modelos. A resposta sobre procedimentos é restrita a esse material. O projeto não possui manuais de fabricante nem histórico de manutenção; a ausência está documentada e faz parte do terceiro cenário de teste.

A busca pode omitir trechos relevantes quando a pergunta usa palavras diferentes. A geração pode interpretar incorretamente uma fonte mesmo quando seu identificador existe. O gráfico da interface resume a série para exibição; os totais usam todas as leituras filtradas.

Prioridade	Próxima evolução
Alta	Adicionar manuais e registros de manutenção autorizados, com identificação do ativo e versão.
Alta	Avaliar dados reais e respostas com especialistas, antes de uso industrial.
Alta	Ampliar testes de sustentação das respostas, informação ausente e ataques ao contexto.
Média	Comparar a busca lexical com busca semântica e medir a recuperação em corpus maior.
Média	Rever limiares e severidade conforme os custos reais de erros e alertas.
Média	Adicionar autenticação e ingestão contínua somente se o projeto evoluir além do uso local.

10. Organização e conferência da entrega

A entrega é composta por este PDF e pelo repositório público no GitHub, que reúne o material necessário à execução e à análise técnica. O README apresenta a visão geral, a estrutura e os comandos. Os modelos são preparados localmente pelo código disponível no repositório.

https://github.com/betitanx/forzy-sprint-04

Requisito	Local de conferência
1. Visão geral	Seção 1.
2. Dados	Seção 2 e evidencias/qualidade_dados.json.
3. Arquitetura	Seção 3 e módulos do projeto.
4. Modelos	Seção 4, métricas e reprodução registrada.
5. RAG	Seção 5, recuperacao.py e assistente.py.
6. Assistente	Seção 6 e interface web.
7. Testes	Seção 7 e pasta evidencias/.
8. Execução	Seção 8, README e preparar_projeto.py.
9. Limitações	Seção 9.
10. Organização	Este PDF e o repositório público.

A revisão conclui que os dez itens estão documentados para o escopo implementado. As limitações não foram transformadas em funcionalidades fictícias.

Referências externas: documentação oficial OpenAI sobre saídas estruturadas em developers.openai.com/api/docs/guides/structured-outputs e chamada de funções em developers.openai.com/api/docs/guides/function-calling. As fontes operacionais e de domínio são os arquivos fornecidos nas sprints, identificados no projeto.